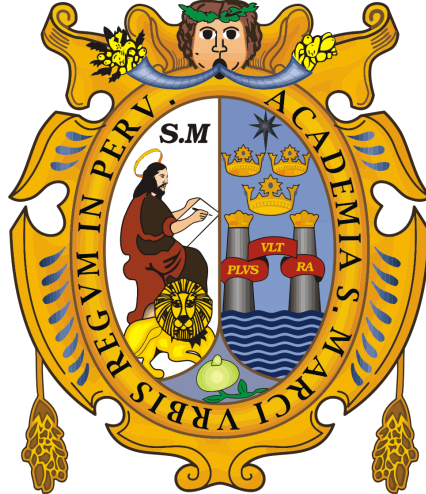


"Año de la Esperanza y el Fortalecimiento de la Democracia"

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS
(Universidad del Perú, Decana de América)



FACULTAD DE INGENIERÍA DE SISTEMAS E INFORMÁTICA

MONOGRAFÍA: COMPUTACIÓN CUÁNTICA

Curso:

Introducción a la Computación

Docente:

Uscuchagua Flores, Gelber Christian

Integrantes:

Angeles, Gian Franco

Eneque Roca, Jair Stefano

Espinal Bazan, Sebastián Andres

Fernandez Curas, Christian Jair

LIMA - PERÚ
2026

Índice

I	Introducción	2
II	Desarrollo	2
2.1	Capítulo I. Principios y Fundamentos de la Computación Cuántica	2
2.1.1	Evolución de la computación clásica a la computación cuántica . . .	2
2.1.2	El qubit como unidad fundamental de información	3
2.1.3	Principios de la mecánica cuántica aplicados a la computación . . .	4
2.1.4	Puertas lógicas y circuitos cuánticos	5
2.1.5	Tecnologías de hardware cuántico	5
2.1.6	Estado actual del hardware cuántico	7
2.2	Capítulo II. Aplicaciones Diarias, Limitaciones y Contraste (Perú vs. Mundo)	10
2.2.1	Ciberseguridad y la amenaza cuántica	10
2.2.2	Convergencia tecnológica: Salud e Inteligencia Artificial	10
2.2.3	Limitaciones físicas y el desafío energético	10
2.2.4	Contraste geopolítico: Hegemonía global frente al reto de adopción en el Perú	10
2.3	Capítulo III. Lógica Cuántica, Programación y Simuladores	10
2.3.1	Simuladores Cuánticos y Entornos de Desarrollo: El Ecosistema IBM Quantum	10
2.3.2	Fundamento teórico de los algoritmos implementados	11
2.3.3	Implementación Práctica: API Cuántica con Qiskit y FastAPI . . .	16
2.3.4	Código fuente del orquestador cuántico	16
2.3.5	Análisis de la Arquitectura y Ejecución	20
2.4	Capítulo IV. Implicaciones Multidisciplinarias	20
2.4.1	El cambio de paradigma filosófico y el fin del determinismo	20
2.4.2	El existencialismo y los multiversos en la literatura	20
2.4.3	Arte, composición musical y la verdadera aleatoriedad	20
III	Conclusiones	20

I Introducción

II Desarrollo

2.1 Capítulo I. Principios y Fundamentos de la Computación Cuántica

2.1.1 Evolución de la computación clásica a la computación cuántica

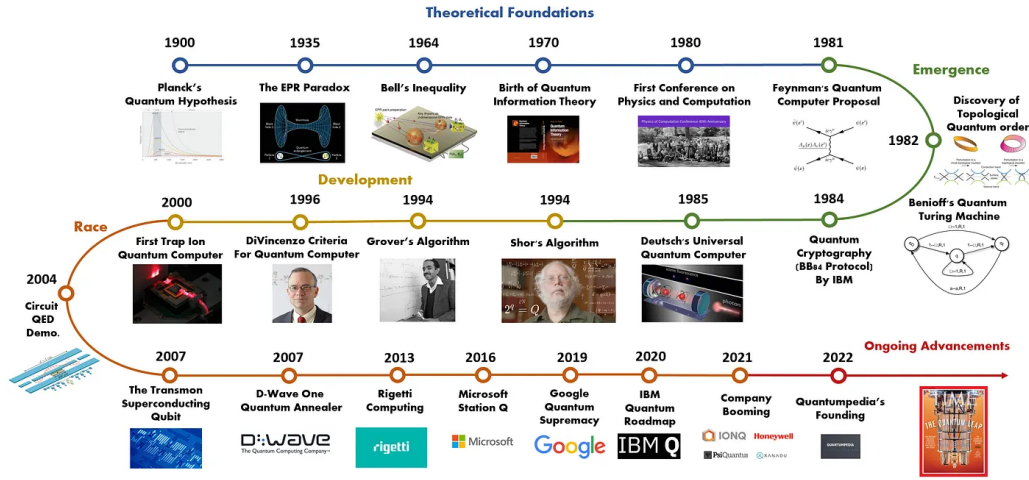
La computación clásica, cuyas bases lógicas y teóricas fueron formalizadas por Alan Turing y otros investigadores en la década de 1930, se fundamenta en bits binarios que toman valores discretos de 0 o 1 y en operaciones lógicas ejecutadas por circuitos electrónicos basados en el álgebra de Boole. Durante décadas, este paradigma guió el desarrollo informático bajo la premisa subyacente de que la computación es un proceso físico sujeto a leyes termodinámicas y naturales. No obstante, las limitaciones intrínsecas de los sistemas convencionales para simular sistemas de la física cuántica de manera eficiente se hicieron evidentes hacia finales del siglo XX.

El surgimiento formal de la computación cuántica se atribuye a una célebre conferencia dictada por Richard Feynman en 1982, el cual imaginó teóricamente una máquina que pudiera imitar y simular de forma fidedigna la mecánica cuántica utilizando los propios principios de la física cuántica (Y. Ozhigov, 2023). Además, esta visión fue formalizada por David Deutsch en 1985 con el concepto de la Máquina de Turing Cuántica, demostrando que tales dispositivos podrían realizar tareas inalcanzables para los sistemas clásicos. Por lo cual, el impulso científico global hacia la construcción de este hardware se aceleró críticamente en 1994, cuando Peter Shor demostró matemáticamente que una computadora cuántica podría factorizar números enteros de forma eficiente, comprometiendo de manera directa los criptosistemas de clave pública modernos como la encriptación RSA.

Como se puede apreciar en la Figura 1, la transición desde los fundamentos computacionales clásicos hasta la consolidación de la teoría cuántica no fue un evento aislado, sino una concatenación de hitos conceptuales. La cronología resalta cómo los desafíos teóricos planteados inicialmente por Feynman y Deutsch encontraron una aplicación crítica y disruptiva con el algoritmo de Shor. Este recorrido histórico demuestra que el paso del bit al qubit no solo representó una mejora en la velocidad de procesamiento, sino un cambio radical en las leyes físicas aplicadas a la teoría de la información, sentando las bases tecnológicas del hardware contemporáneo.

Figura 1

Línea de tiempo de la evolución histórica de la computación cuántica



Nota. Adaptado de *A Brief History of Quantum Computing*, por Quantumpedia, 2020 (<https://quantumpedia.uk/a-brief-history-of-quantum-computing-e0bbd05893d0>).

2.1.2 El qubit como unidad fundamental de información

Mientras que la arquitectura clásica procesa la información mediante el bit convencional, la computación cuántica introduce el bit cuántico o *qubit* como su bloque elemental y unidad fundamental de información. Físicamente, un qubit se define como un sistema cuántico direccionable de dos niveles de energía.

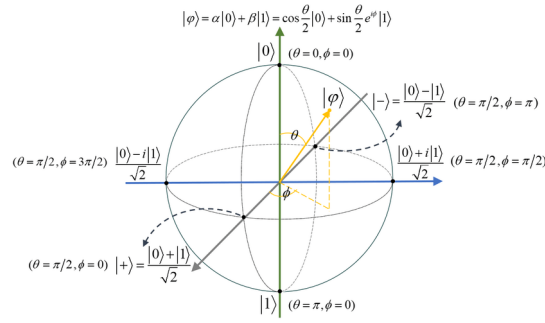
Matemáticamente, el estado puro de un qubit se modela como un vector de estado que reside dentro de un espacio de Hilbert complejo bidimensional. Haciendo uso de la notación bra-ket de Dirac, el estado general de un qubit, denotado como $|\psi\rangle$, se expresa formalmente a través de la combinación lineal de sus estados de base computacional, $|0\rangle$ y $|1\rangle$, definida por la ecuación:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \quad (1)$$

donde α y β son amplitudes de probabilidad complejas que cumplen la condición de normalización $|\alpha|^2 + |\beta|^2 = 1$. Esta propiedad permite que n qubits representen simultáneamente 2^n valores posibles, expandiendo el espacio computacional (espacio de Hilbert) de manera exponencial. Físicamente, los qubits pueden implementarse mediante el espín de un electrón, la polarización de un fotón o estados de carga en circuitos superconductores.

Figura 2

Representación geométrica de un qubit en la esfera de Bloch



Nota. Tomado de *Efficient Quantum Secure Vector Dominance and Its Applications in Computational Geometry*, por W. Liu, B. Su y F. Sun, 2025, *IEEE Transactions on Computers*, 74(6), pp. 2129–2143 (<https://doi.org/10.1109/TC.2025.3557968>).

Tal y como se puede visualizar en la Figura 2, la esfera de Bloch proporciona una abstracción geométrica indispensable para visualizar el estado de un qubit aislado dentro de un espacio tridimensional. Mientras que un bit clásico está limitado a los polos norte o sur de esta esfera (representando estrictamente los estados discretos $|0\rangle$ y $|1\rangle$), un qubit puede localizarse en cualquier punto sobre la superficie de la esfera. Los ángulos colatitud y longitud de la superficie parametrizan las amplitudes complejas α y β , lo que evidencia de forma visual cómo el principio de superposición permite la coexistencia e infinitas combinaciones de estados. Esta propiedad geométrica es fundamental para el diseño de algoritmos, ya que las operaciones lógicas cuánticas equivalen analíticamente a rotaciones controladas alrededor de los ejes de este espacio esférico.

2.1.3 Principios de la mecánica cuántica aplicados a la computación

La ventaja de procesamiento radical que exhibe el paradigma cuántico sobre los enfoques digitales clásicos radica en la explotación directa de los postulados fundamentales de la mecánica cuántica. Los principios esenciales integrados en el procesamiento de información son los siguientes:

- **Superposición cuántica:** Es la propiedad física que faculta a un sistema (como el qubit) para coexistir simultáneamente en múltiples estados combinados ($|0\rangle$ y $|1\rangle$) con amplitudes de probabilidad definidas, en lugar de verse confinado de forma exclusiva a una única alternativa binaria rígida.
- **Entrelazamiento cuántico:** Fenómeno físico en el cual los estados cuánticos de dos o más partículas u objetos se vinculan de tal manera que el estado de uno de ellos no puede describirse de forma independiente del otro, sin importar la distancia física que los separe. El entrelazamiento genera correlaciones no locales masivas y un procesamiento en paralelo inalcanzable por medios convencionales.

- **Interferencia:** Se utiliza en los algoritmos para amplificar las amplitudes de probabilidad de los resultados correctos y cancelar mediante interferencia destructiva los resultados erróneos.

Un aspecto crítico es la medición, la cual colapsa el estado de superposición en un resultado determinista (0 o 1), destruyendo la información cuántica previa. Además, el teorema de no clonación prohíbe la creación de copias idénticas de un estado cuántico desconocido, lo que diferencia radicalmente la gestión de errores cuánticos de la clásica (Nasser, 2025, p.5).

2.1.4 Puertas lógicas y circuitos cuánticos

Las operaciones en computación cuántica se ejecutan mediante **puertas lógicas cuánticas**, que son transformaciones unitarias aplicadas a los qubits. A diferencia de las puertas clásicas, las cuánticas son inherentemente reversibles.

Entre las puertas fundamentales se encuentran:

- **Puerta Hadamard (H):** Crea superposición a partir de estados base.
- **Puertas de Pauli (X, Y, Z):** Realizan rotaciones en la esfera de Bloch, como la puerta X que actúa como un *NOT* cuántico.
- **Puerta CNOT (Controlled-NOT):** Una puerta de dos qubits esencial para generar entrelazamiento.

Un circuito cuántico es una secuencia de estas puertas aplicadas a un estado inicial para realizar un cálculo específico. Asimismo, se ha demostrado que conjuntos pequeños de estas puertas son universales, lo que significa que cualquier operación unitaria puede aproximarse con precisión arbitraria mediante una secuencia finita de ellas (Madanan et al., 2023).

2.1.5 Tecnologías de hardware cuántico

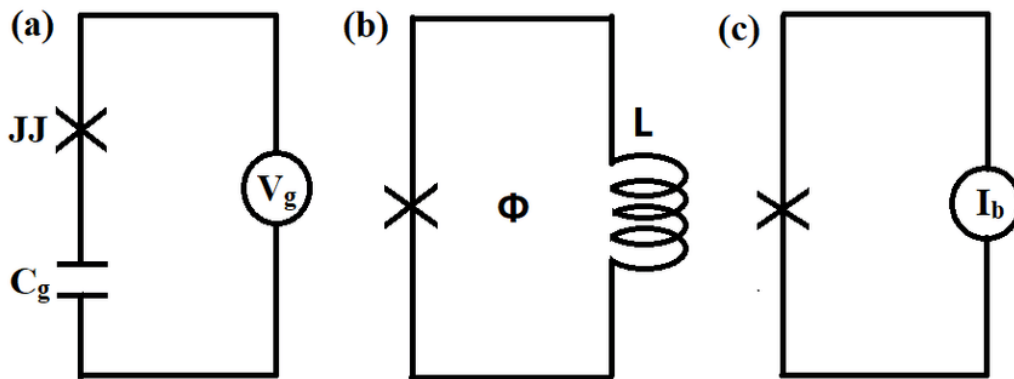
La materialización física de un ordenador cuántico escalable requiere de un sistema tecnológico capaz de satisfacer rigurosamente los denominados criterios de DiVincenzo formalizados en el año 2000. Dichos criterios exigen cinco directrices fundamentales para el procesamiento: (a) un sistema físico escalable con qubits bien caracterizados, (b) la capacidad de inicializar los estados de los qubits, (c) tiempos largos de coherencia cuántica, (d) un conjunto universal de compuertas cuánticas, y (e) la capacidad de realizar mediciones específicas en qubits individuales (He-Liang Huang et al., 2020). Actualmente, en el siglo XXI, coexisten múltiples plataformas de hardware en desarrollo activo:

- **Circuitos Superconductores:** Constituyen circuitos electrónicos de estado sólido fabricados mediante microtecnología semiconductora estándar. Operan manipulando grados de libertad como la carga, el flujo magnético o la fase cuántica a través de uniones Josephson. Ofrecen una gran facilidad de acoplamiento, alta flexibilidad de diseño y operaciones lógicas extremadamente rápidas, aunque demandan refrigeración extrema a temperaturas de entre 10 y 20 milikelvins mediante refrigeradores de dilución.
- **Iones Atrapados:** Utilizan átomos individuales cargados eléctricamente (iones) que son suspendidos y confinados en el vacío mediante campos electromagnéticos generados por trampas Paul de radiofrecuencia. La información se codifica en sus niveles hiperfinos y las operaciones lógicas se ejecutan mediante la aplicación dirigida de pulsos láser. Sobresalen por exhibir tiempos de coherencia significativamente prolongados.
- **Sistemas Fotónicos:** Emplean partículas individuales de luz (fotones) como qubits itinerantes manipulados por medio de componentes ópticos lineales (divisores de haz, guías de onda y desfasadores). Presentan una resistencia intrínseca casi total a la decoherencia térmica ambiental, pero afrontan la severa dificultad de lograr interacciones deterministas eficientes entre fotones independientes.
- **Qubits de Espín en Estado Sólido:** Codifican la información en el espín de electrones atrapados dentro de puntos cuánticos semiconductores o en defectos estructurales cristalinos, como los centros de vacante de nitrógeno (NV) en diamantes, ideales para arquitecturas híbridas de memoria cuántica.

Para ilustrar físicamente los esquemas fundamentales de los circuitos de estado sólido descritos, en la Figura 3 se presenta el diagrama conceptual de las tres tipologías básicas de qubits superconductores.

Figura 3

Diagramas esquemáticos de las tres tipologías principales de qubits superconductores



Nota. (a) Qubit de carga compuesto por una unión Josephson (JJ) y un capacitor con voltaje de compuerta V_g . (b) Qubit de flujo que muestra la inductancia de lazo L y el flujo de polarización Φ . (c) Qubit de fase con corriente de polarización I_b . Tomado de *Superconducting quantum computing: a review*, por H.-L. Huang, D. Wu, D. Fan et al., 2020, *Science China Information Sciences*, 63, Artículo 180501 (<https://doi.org/10.1007/s11432-020-2881-9>).

Como se detalla en la Figura 3, el comportamiento cuántico en estos circuitos superconductores se logra aislando diferentes variables macroscópicas a través del uso de uniones Josephson (representadas con una 'X'). En el qubit de carga (a), el estado se define mediante el número de pares de Cooper en la isla cuántica controlada por el voltaje V_g . Por otro lado, el qubit de flujo (b) opera basándose en la dirección de la corriente persistente y el flujo magnético Φ que atraviesa el lazo inductivo. Finalmente, el qubit de fase (c) aprovecha los niveles de energía cuántica oscilatorios dentro del pozo de potencial modificado por una corriente de polarización externa I_b . Esta diversidad en el diseño demuestra el alto grado de flexibilidad técnica que poseen las plataformas superconductoras para modelar sistemas artificiales de dos niveles de energía.

2.1.6 Estado actual del hardware cuántico

En este siglo XXI, nos encontramos en la era de los Sistemas Cuánticos de Escala Intermedia con Ruido (NISQ). Estos dispositivos cuentan con entre 50 y unos pocos cientos de qubits, pero carecen de una corrección de errores completa, lo que limita la profundidad de los circuitos debido a la decoherencia y el ruido ambiental (Singh Gill et al., 2020).

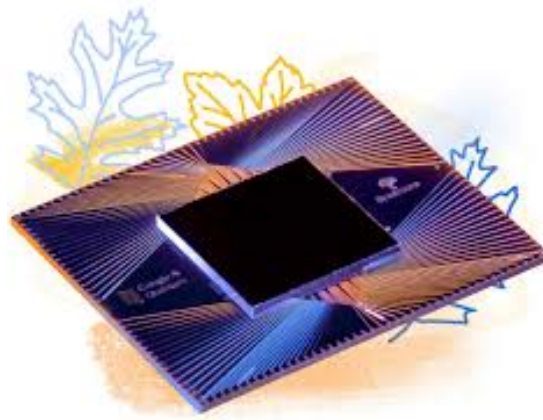
Un hito histórico fundamental dentro de esta etapa fue la demostración práctica de la supremacía cuántica en 2019 por parte de Google, cuyo procesador *Sycamore* de 53 qubits ejecutó en un lapso de segundos una tarea de muestreo que le habría tomado milenios resolver a una supercomputadora convencional. En consecuencia, el desafío científico contemporáneo se centra en transicionar de esta supremacía teórica hacia una ventaja cuántica.

tica práctica orientada a la resolución de problemas del mundo real en campos altamente complejos como la criptografía, la ciencia de materiales y la optimización combinatoria.

Con el fin de desarrollar sistemas comerciales a gran escala, los esfuerzos de investigación se dirigen prioritariamente hacia la implementación de códigos de corrección de errores cuánticos, tales como el código de superficie (*surface code*), diseñados de forma específica para blindar y preservar la información en arquitecturas de hardware inherentemente ruidosas.

Figura 4

Procesador cuántico superconductor de escala intermedia de la familia Google Quantum AI

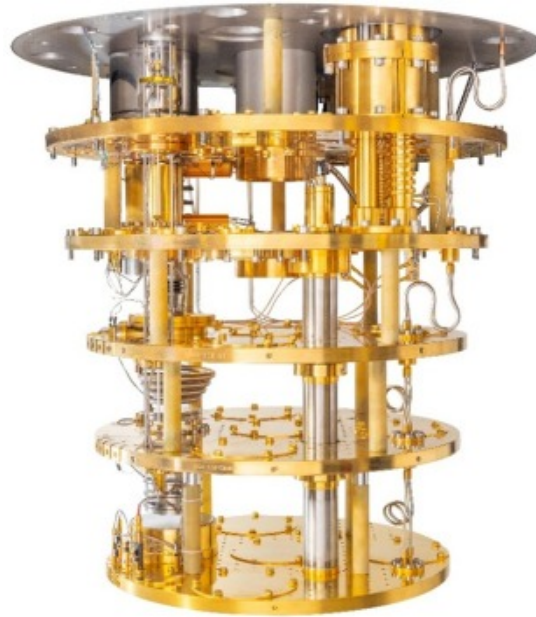


Nota. Vista macroscópica del empaquetamiento y líneas de acoplamiento de un chip cuántico superconductor diseñado para la ejecución de algoritmos a través de compuertas lógicas unitarias programables. Adaptado de *Quantum Computer Datasheet: Weber*, por Google Quantum AI, s.f. (<https://quantumai.google/hardware/datasheet/weber.pdf>).

Como se describió al analizar las exigencias de las plataformas superconductoras y los dispositivos de la era NISQ, estos procesadores requieren entornos térmicos extremadamente estables y fríos para operar. Para mantener la coherencia cuántica en los qubits y blindar el sistema frente a las interacciones disruptivas del entorno, se utilizan sistemas de criogenia altamente sofisticados conocidos como refrigeradores de dilución, cuya estructura interna se detalla en la Figura 5.

Figura 5

Infraestructura interna de un refrigerador de dilución utilizado para la criogenia de hardware cuántico superconductor



Nota. Vista detallada de las placas de brida doradas y las etapas de enfriamiento progresivo que permiten aislar los sistemas del ruido térmico ambiental mediante temperaturas en el rango de los milikelvins. Tomado de *Development of dilution refrigerators—A review*, por H. Zu, W. Dai y A. T. A. M. de Waele, 2022, *Cryogenics*, 121, Artículo 103390 (<https://doi.org/10.1016/j.cryogenics.2021.103390>).

El uso de los refrigeradores de dilución es indispensable debido a que la energía de los enlaces térmicos a temperatura ambiente destruiría instantáneamente los delicados estados de superposición y entrelazamiento de los circuitos. Mediante la mezcla controlada de los isótopos de helio-3 y helio-4, estas cámaras logran mitigar la agitación térmica de los electrones en los materiales superconductores, garantizando que las fluctuaciones del entorno físico exterior no interfieran con las operaciones unitarias programadas en los algoritmos del procesador.

2.2 Capítulo II. Aplicaciones Diarias, Limitaciones y Contraste (Perú vs. Mundo)

2.2.1 Ciberseguridad y la amenaza cuántica

2.2.2 Convergencia tecnológica: Salud e Inteligencia Artificial

2.2.3 Limitaciones físicas y el desafío energético

2.2.4 Contraste geopolítico: Hegemonía global frente al reto de adopción en el Perú

2.3 Capítulo III. Algoritmia Cuántica, Entornos de Desarrollo e Implementación Híbrida

2.3.1 Simuladores Cuánticos y Entornos de Desarrollo: El Ecosistema IBM Quantum

La programación cuántica difiere del paradigma tradicional al sustituir las operaciones lógicas de asignación binaria discreta por la evolución unitaria de vectores de estado en espacios complejos. Para manipular la información contenida en los qubits, se diseñan circuitos basados en compuertas cuánticas elementales representadas matemáticamente por matrices unitarias continuas. Debido a que el hardware físico contemporáneo de la era de los sistemas cuánticos de escala intermedia con ruido presenta altas tasas de error por la agitación térmica y la decoherencia cuántica, la comunidad científica depende estrictamente de simuladores cuánticos ejecutados en hardware clásico. Actualmente, el mercado tecnológico ofrece diversas plataformas competitivas, tales como el Quantum Development Kit de Microsoft, Cirq de Google o Braket de Amazon. Sin embargo, la adopción del ecosistema de IBM Quantum y su framework de código abierto Qiskit se ha posicionado como el estándar de la industria y la academia.

La selección del entorno de IBM para el desarrollo del orquestador cuántico en este proyecto se fundamenta en tres ventajas arquitectónicas críticas. En primer lugar, la accesibilidad y democratización mediante el esquema de computación en la nube permite el acceso remoto a computadoras cuánticas físicas reales, eliminando las barreras de infraestructura y facilitando la investigación en el Perú sin necesidad de poseer hardware local. En segundo lugar, la emulación precisa de ruido provista por la librería AerSimulator faculta el modelado del comportamiento ideal de un circuito y la inyección de patrones de agitación térmica para mitigar errores antes de la ejecución real. En tercer lugar, la orquestación híbrida a través de Qiskit Runtime permite entrelazar dinámicamente recursos clásicos en Python con rutinas de procesamiento cuántico pesado dentro de una sesión dedicada, disminuyendo críticamente la latencia de cola. Gracias a este ecosistema, los

ingenieros de software pueden modelar, depurar y validar sus circuitos de manera local antes de delegar la compilación definitiva.

2.3.2 Fundamento teórico de los algoritmos implementados

El sistema desarrollado está diseñado para orquestar la ejecución de tres de los algoritmos más representativos de la ventaja cuántica, estructurados en un catálogo de casos de estudio dentro del orquestador.

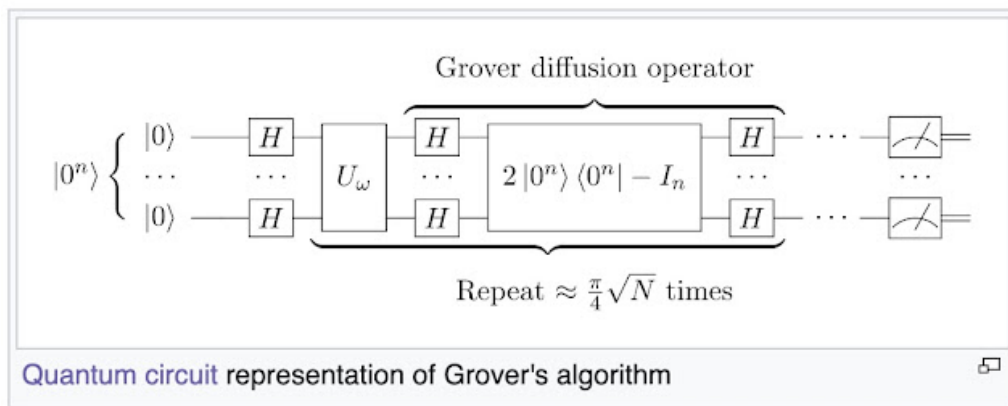
El primer pilar lo constituye el Algoritmo de Grover, enfocado en la búsqueda no estructurada. En la computación clásica, localizar un elemento en una base de datos de tamaño N requiere una complejidad temporal de $\mathcal{O}(N)$, mientras que el enfoque cuántico reduce este coste a $\mathcal{O}(\sqrt{N})$ mediante la inversión sobre la media para amplificar la amplitud del estado correcto. Su ejecución matemática inicia con la preparación del sistema en una superposición uniforme de todos los estados posibles, expresada mediante la ecuación:

$$|s\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle$$

Posteriormente, se aplica una secuencia repetitiva donde un operador oráculo invierte la fase del estado objetivo y un operador de difusión de Grover ejecuta la inversión matricial definida como $U_s = 2|s\rangle\langle s| - I$. Este ciclo se reitera un número óptimo de veces acotado por la expresión $r(N) \leq \lceil \frac{\pi}{4}\sqrt{N} \rceil$ antes de proceder con la medición final en la base computacional.

Figura 6

Representación del Algoritmo de Grover en un Circuito Cuántico



Nota. Esquema representativo de las iteraciones de oráculo y difusión en un circuito. Adaptado de "A fast quantum mechanical algorithm for database search".

Para ilustrar estos conceptos matemáticos en nuestra implementación práctica, se ha desarrollado un circuito base de 3 qubits que representa la lógica de Grover en el Caso 0 de

la API. A continuación, se presenta el fragmento de código que combina el procesamiento cuántico en Qiskit con la extracción clásica de datos:

Código 1: Ansatz de Grover y extracción híbrida en Python

```
1  # Caso 0: Grover (Busqueda en base de datos CSV)
2  if req.case_id == 0:
3      # 1. Inicializacion: Superposicion uniforme
4      qc.h([0, 1, 2])
5
6      # 2. Oraculo: Marca el estado objetivo invirtiendo la fase
7      qc.cz(0, 2)
8
9      # 3. Operador de Difusion (Ansatz simplificado)
10     qc.h([0, 1, 2])
11     algoritmo = "Grover"
12
13     # 4. Mapeo Clasico: Uso del indice resultante devuelto por IBM
14     if req.target_index is not None:
15         archivo_csv = "usuarios_masivos.csv"
16         if os.path.exists(archivo_csv):
17             with open(archivo_csv, mode='r', encoding='utf-8') as file:
18                 for i, linea in enumerate(file):
19                     if i == req.target_index:
20                         fila = linea.strip().replace(",", " | ")
21                         resultado_especial = f"{req.target_index}, {fila}"
22                         print(f"[OK] Fila extraida localmente: ID {req.target_index}")
23                         break
```

La implementación refleja un enfoque híbrido cuántico-clásico. En las primeras líneas, el circuito aplica compuertas de Hadamard a todos los registros, cumpliendo la etapa de inicialización teórica. Posteriormente, se aplica una compuerta controlada Z que actúa como el oráculo, marcando el estado. El segundo bloque de compuertas Hadamard inicia la difusión para amplificar la probabilidad del estado marcado. Una vez que la Unidad de Procesamiento Cuántico de IBM colapsa el estado mediante la medición y devuelve el resultado, el algoritmo transiciona a la lógica clásica en Python. El sistema utiliza dicho índice cuántico para buscar y extraer información directamente en una base de datos real (el archivo `usuarios_masivos.csv`), validando empíricamente cómo el cálculo probabilístico acelera la búsqueda. A nivel de diseño de circuitos, la implementación de estos pasos exige una cantidad de compuertas lógicas lineal respecto al número de qubits, lo que resulta en una complejidad de $\mathcal{O}(\log N)$ por iteración.

El segundo pilar está definido por el Algoritmo de Shor, enfocado en la factorización

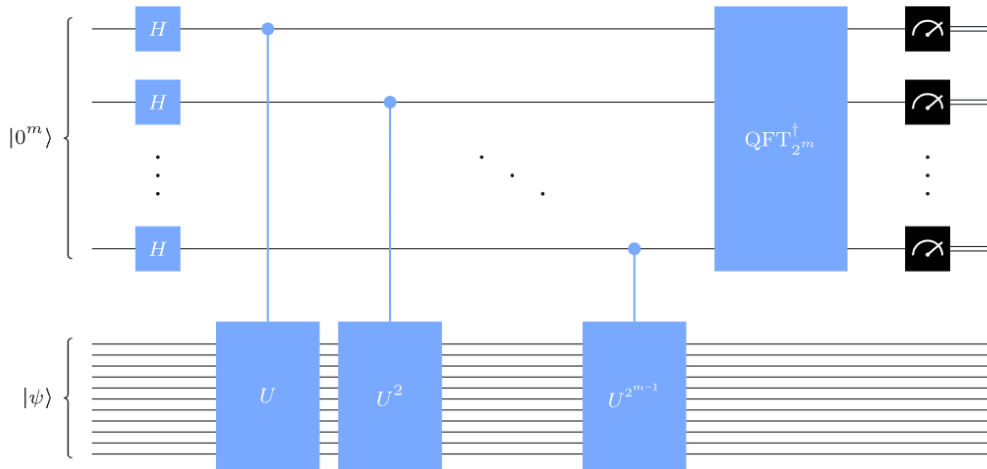
matemática y el criptoanálisis. Este esquema resuelve la descomposición de números enteros gigantes en factores primos en tiempo polinómico $\mathcal{O}((\log N)^3)$, quebrando esquemas de clave pública como RSA que dependen de la intratabilidad clásica exponencial del problema. El proceso inicia con una reducción clásica que elige un entero aleatorio coprimo con N para estructurar la función modular $f(x) = a^x \pmod{N}$. La búsqueda cuántica del periodo r de dicha función constituye el núcleo del algoritmo, donde un operador de exponenciación modular entrelaza los registros en el estado $|x\rangle|a^x \pmod{N}\rangle$, seguido por la aplicación de la Transformada de Fourier Cuántica para revelar las frecuencias periódicas mediante la expresión:

$$|x\rangle \xrightarrow{QFT} \frac{1}{\sqrt{Q}} \sum_{y=0}^{Q-1} e^{2\pi i \frac{xy}{Q}} |y\rangle$$

La medición colapsa el sistema en un valor que permite deducir el periodo mediante fracciones continuas, calculando finalmente los factores primos clásicamente con el máximo común divisor $\gcd(a^{r/2} \pm 1, N)$.

Figura 7

Arquitectura del Circuito Cuántico para el Algoritmo de Shor



Nota. Diagrama esquemático que ilustra los registros de inicialización, la compuerta de exponenciación modular y la Transformada de Fourier Cuántica inversa. Adaptado de "Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer".

Dado que los dispositivos ruidosos actuales carecen del volumen de qubits lógicos necesarios para aplicar la Transformada de Fourier Cuántica sobre claves reales, en nuestra implementación de la API (Caso 1) se emplea un enfoque de emulación funcional. El circuito cuántico prepara el estado base, mientras que el backend clásico simula la extracción de los factores hallados por la rutina para ejecutar el quiebre criptográfico.

Código 2: Orquestación del Algoritmo de Shor y Ataque RSA

```

1  # Funcion auxiliar: Simulacion de la extraccion de periodo (Factores p y
    q)
2  def simular_shor_factorizacion(n):
3      if n % 2 == 0: return 2, n // 2
4      for i in range(3, int(math.isqrt(n)) + 1, 2):
5          if n % i == 0:
6              return i, n // i
7      return None, None
8
9  # Caso 1: Shor (Ataque RSA a traves de API)
10 elif req.case_id == 1:
11     # 1. Preparacion del estado cuantico inicial (Ansatz base)
12     qc.h([0, 1, 2])
13     algoritmo = "Shor"
14     print("[*] Iniciando simulacion de ataque RSA...")
15
16     # 2. Recepcion de la clave publica (N, e) y el mensaje cifrado
17     if req.n_rsa and req.e_rsa and req.cipher_rsa:
18         # 3. Simulacion de la busqueda de periodo de Shor
19         p, q = simular_shor_factorizacion(req.n_rsa)
20
21         if p and q:
22             # 4. Quiebre criptografico: Calculo de la Totiente de Euler
23             phi = (p - 1) * (q - 1)
24             # Generacion de la clave privada d
25             d = pow(req.e_rsa, -1, phi)
26
27             # 5. Desencryptacion del mensaje
28             decrypted_chars = [safe_chr(pow(c, d, req.n_rsa)) for c in
                                req.cipher_rsa]
29             texto_descifrado = "".join(decrypted_chars)
30
31             resultado_especial = f"HACKEADO! Factores: p={p}, q={q} |
                                    Mensaje: '{texto_descifrado}'"
32             print("[OK] Mensaje desencriptado exitosamente.")

```

El análisis de la rutina evidencia la vulnerabilidad estructural de la criptografía asimétrica. Al obtener los factores primos mediante el procedimiento simulado en la función dedicada, el orquestador es capaz de derivar la función totiente de Euler $\phi(N) = (p-1)(q-1)$ y calcular la clave privada d mediante el inverso multiplicativo modular del exponente público. La reversión final del criptograma confirma la fragilidad de los sistemas tradicionales frente al cómputo cuántico.

El tercer pilar lo compone el Algoritmo de Optimización Aproximada Cuántica, dis-

añado como un enfoque iterativo híbrido para resolver problemas combinatorios de alta complejidad en la era de los procesadores con ruido ambiental, modelando escenarios como el problema del corte máximo o el Problema del Viajante. El problema clásico se expresa originalmente como una optimización binaria donde cada nodo toma una variable discreta, buscando maximizar las conexiones que cruzan la partición mediante la función:

$$\max_{x \in \{0,1\}^n} \sum_{(i,j)} x_i + x_j - 2x_i x_j$$

Este modelo se traduce a una optimización binaria cuadrática no restringida de la forma $\min x^T Q x$ y, mediante la sustitución matemática de variables $x_i \rightarrow (1 - Z_i)/2$, se mapea de forma directa a un Hamiltoniano de coste basado en operadores espín de Pauli Z :

$$H_C = \sum_{ij} Q_{ij} Z_i Z_j + \sum_i b_i Z_i$$

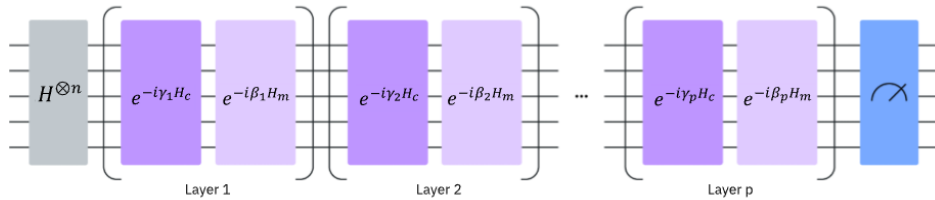
Para grafos no pesados, los coeficientes lineales se anulan, fijando la función objetivo. El circuito prepara los estados aplicando capas alternadas de un operador de coste $e^{-i\gamma_k H_C}$ y un operador mezclador $e^{-i\beta_k H_m}$ sobre un estado inicial de superposición uniforme, estructurando analíticamente la ecuación de evolución unitaria:

$$|\psi(\boldsymbol{\gamma}, \boldsymbol{\beta})\rangle = \prod_{k=1}^p e^{-i\beta_k H_m} e^{-i\gamma_k H_C} |s\rangle$$

Finalmente, el sistema mide el estado para evaluar el coste esperado y delega los resultados a un optimizador clásico encargado de actualizar los ángulos en un bucle de retroalimentación cerrado dentro de una sesión de Qiskit Runtime.

Figura 8

Estructura de Capas Alternadas en el Algoritmo QAOA



Nota. Diagrama esquemático que ilustra la aplicación secuencial de los operadores de coste y mezcla sobre el estado inicial de superposición. Adaptado de la documentación oficial de Qiskit sobre optimización aproximada.

En nuestra implementación correspondiente al Caso 2 de la API, se prepara la estructura base del circuito para la evaluación de rutas logísticas. A continuación, se detalla el fragmento de código que define el Ansatz y gestiona la ejecución en el hardware real:

Código 3: Preparación del Ansatz QAOA y orquestación híbrida

```

1  # Caso 2: QAOA (Optimización TSP)
2  elif req.case_id == 2:
3      # 1. Aplicación de rotaciones parametrizadas y entrelazamiento
4      qc.rx(math.pi/2, [0, 1, 2])
5      qc.cx(0, 1)
6      qc.cx(0, 1)
7      algoritmo = "QAOA"
8
9  # 2. Medición del estado resultante
10 qc.measure([0, 1, 2], [0, 1, 2])
11
12 # 3. Ejecución en IBM Cloud (Bucle híbrido)
13 try:
14     print("[WAIT] Conectando con IBM Cloud...")
15     service = QiskitRuntimeService(channel="ibm_cloud", token=
16         API_KEY_CLOUD, instance=CRN_INSTANCIA)
17     backend_real = service.least_busy(operational=True, simulator=False)
18
19     pm = generate_preset_pass_manager(backend=backend_real,
20         optimization_level=1)
21     circuito_isa = pm.run(qc)
22
23     sampler = Sampler(mode=backend_real)
24     job = sampler.run([circuito_isa])
25     conteos = job.result()[0].data.c.get_counts()
26     estado_final = max(conteos, key=conteos.get)
27     print(f"[OK] Estado optimo hallado: {estado_final}")

```

El análisis de la arquitectura híbrida ilustra la sinergia que define al algoritmo. El framework clásico en Python actúa como el coordinador principal, configurando un circuito con rotaciones en el eje X y compuertas de entrelazamiento condicionado CNOT para mapear los parámetros. Al despacharse el trabajo a la infraestructura física de IBM Quantum, el procesador evalúa simultáneamente las combinaciones probabilísticas y devuelve el histograma de distribución, aislando la cadena binaria óptima mediante una función clásica de selección de máximos.

2.3.3 Implementación Práctica: API Cuántica con Qiskit y FastAPI

Para demostrar la viabilidad técnica de la computación cuántica en arquitecturas de software modernas, se ha desarrollado un *backend* utilizando el framework FastAPI en Python, integrado con el SDK Qiskit de IBM. Esta arquitectura permite la ejecución de rutinas cuánticas bajo demanda, delegando el procesamiento pesado a las Unidades de Procesamiento Cuántico (QPU) de IBM Cloud o, en su defecto, a simuladores locales.

2.3.4 Código fuente del orquestador cuántico

A continuación, se detalla el código fuente del microservicio encargado de enrutar las peticiones al hardware de IBM:

Código 4: Orquestador API Cuántico con Qiskit Runtime

```
1 from fastapi import FastAPI
2 from fastapi.middleware.cors import CORSMiddleware
3 from pydantic import BaseModel
4 import math
5 import csv
6 import os
7 from typing import List, Optional
8
9 from qiskit import QuantumCircuit, transpile
10 from qiskit_ibm_runtime import QiskitRuntimeService, SamplerV2 as
    Sampler
11 from qiskit.transpiler.preset_passmanagers import import
    generate_preset_pass_manager
12 from qiskit_aer import AerSimulator
13
14 app = FastAPI()
15
16 app.add_middleware(
17     CORSMiddleware,
18     allow_origins=["*"],
19     allow_methods=["*"],
20     allow_headers=["*"],
21 )
22
23 class CasoRequest(BaseModel):
24     case_id: int
25     n_rsa: Optional[int] = None
26     e_rsa: Optional[int] = None
27     cipher_rsa: Optional[List[int]] = None
28     target_index: Optional[int] = None
29
30 def simular_shor_factorizacion(n):
31     if n % 2 == 0: return 2, n // 2
32     for i in range(3, int(math.isqrt(n)) + 1, 2):
33         if n % i == 0:
34             return i, n // i
35     return None, None
36
37 def safe_chr(val):
38     try:
39         if 0xD800 <= val <= 0xDFFF:
```

```

40         return "?"
41     return chr(val)
42 except ValueError:
43     return "?"
44
45 # Credenciales de IBM Quantum Cloud
46 API_KEY_CLOUD = "2XMZvFEBM3saI3RhSbZQPkVxMaTGdFhzwF0ifZjGS7Gu"
47 CRN_INSTANCIA = "crn:v1:bluemix:public:quantum-computing:us-east:a/
    afff77ba7c6c41688a65aa609d9f7891:5aed1a55-5288-4821-936f-3b5ac997ec86
    ::"
48
49 @app.post("/api/ejecutar-caso")
50 async def ejecutar_caso(req: CasoRequest):
51     print(f"\n{'='*50}\n[INFO] INICIANDO PETICION - CASO {req.case_id +
        1}\n{'='*50}")
52
53     qc = QuantumCircuit(3, 3)
54     algoritmo = ""
55     resultado_especial = ""
56
57     # Caso 0: Grover (Busqueda en base de datos CSV)
58     if req.case_id == 0:
59         qc.h([0, 1, 2])
60         qc.cz(0, 2)
61         qc.h([0, 1, 2])
62         algoritmo = "Grover"
63
64         if req.target_index is not None:
65             archivo_csv = "usuarios_masivos.csv"
66             if os.path.exists(archivo_csv):
67                 with open(archivo_csv, mode='r', encoding='utf-8') as
68                     file:
69                     for i, linea in enumerate(file):
70                         if i == req.target_index:
71                             fila = linea.strip().replace(",", " | ")
72                             resultado_especial = f"{req.target_index}, {
73                                 fila}"
74                             print(f"[OK] Fila extraida localmente: ID {
75                                 req.target_index}")
76                             break
77
78     # Caso 1: Shor (Ataque RSA)
79     elif req.case_id == 1:
80         qc.h([0, 1, 2])
81         algoritmo = "Shor"
82         if req.n_rsa and req.e_rsa and req.cipher_rsa:
83             p, q = simular_shor_factorizacion(req.n_rsa)

```

```

81         if p and q:
82             phi = (p - 1) * (q - 1)
83             d = pow(req.e_rsa, -1, phi)
84             decrypted_chars = [safe_chr(pow(c, d, req.n_rsa)) for c
                                in req.cipher_rsa]
85             texto_descifrado = "".join(decrypted_chars)
86             resultado_especial = f"HACKEADO! Factores: p={p}, q={q}
                                | Mensaje: '{texto_descifrado}'"
87
88         # Caso 2: QAOA (Optimizacion TSP)
89     elif req.case_id == 2:
90         qc.rx(math.pi/2, [0, 1, 2])
91         qc.cx(0, 1)
92         qc.cx(0, 2)
93         algoritmo = "QAOA"
94
95     qc.measure([0, 1, 2], [0, 1, 2])
96     maquina_usada = ""
97     estado_final = ""
98
99     # Integracion y Ejecucion
100    try:
101        print("[WAIT] Conectando con IBM Cloud...")
102        service = QiskitRuntimeService(channel="ibm_cloud", token=
            API_KEY_CLOUD, instance=CRN_INSTANCIA)
103        backend_real = service.least_busy(operational=True, simulator=
            False)
104        maquina_usada = backend_real.name
105
106        pm = generate_preset_pass_manager(backend=backend_real,
            optimization_level=1)
107        circuito_isa = pm.run(qc)
108
109        sampler = Sampler(mode=backend_real)
110        job = sampler.run([circuito_isa])
111        conteos = job.result()[0].data.c.get_counts()
112        estado_final = max(conteos, key=conteos.get)
113
114    except Exception as e:
115        print(f"[ERROR] IBM Cloud no disponible. Activando AerSimulator
            local.")
116        maquina_usada = "AerSimulator (Local)"
117        simulador_local = AerSimulator()
118        circuito_compilado = transpile(qc, backend=simulador_local)
119        job = simulador_local.run(circuito_compilado, shots=1024)
120        conteos = job.result().get_counts()
121        estado_final = max(conteos, key=conteos.get)

```

```

122
123     resultado_final = resultado_especial if resultado_especial else
        estado_final
124
125     return {
126         "status": "success",
127         "algoritmo": algoritmo,
128         "qubit_result": resultado_final,
129         "mensaje": f"Ejecutado via: {maquina_usada}"
130     }

```

2.3.5 Análisis de la Arquitectura y Ejecución

La implementación se estructura bajo un diseño tolerante a fallos que opera en cinco etapas secuenciales perfectamente integradas.

En primer lugar, se establece la capa de abstracción de la API mediante el uso del framework FastAPI y los modelos de validación estructural de Pydantic, configurando una interfaz RESTful capaz de admitir parámetros clásicos de entrada de forma tipada. En segundo lugar, se ejecuta la construcción dinámica del circuito cuántico en memoria empleando la clase QuantumCircuit de Qiskit, aplicando las instrucciones unitarias correspondientes a las transformaciones lógicas deseadas. En tercer lugar, se activa el enrutamiento hacia la nube de IBM Cloud a través del servicio QiskitRuntimeService, autenticando las credenciales y aislando el coprocesador físico de hardware que certifique la menor cola de peticiones activas.

En cuarto lugar, el sistema procesa la transpilación del circuito abstracto mediante un gestor de pases preestablecido, descomponiendo y adaptando las compuertas ideales a la topología nativa de emparejamiento físico del chip cuántico seleccionado. En quinto lugar, se despliega un mecanismo de degradación elegante gobernado por un bloque de control de excepciones, el cual conmuta la orquestación automáticamente hacia el entorno de simulación local AerSimulator ante cualquier interrupción de red o saturación de tokens en la plataforma externa, asegurando la continuidad del servicio de software.

2.4 Capítulo IV. Implicaciones Multidisciplinarias

2.4.1 El cambio de paradigma filosófico y el fin del determinismo

2.4.2 El existencialismo y los multiversos en la literatura

2.4.3 Arte, composición musical y la verdadera aleatoriedad

III Conclusiones

IV. Referencias

- [1] Huang, H.-L., Wu, D., Fan, D., & Zhu, X. (2020). Superconducting quantum computing: a review. *Science China Information Sciences*, 63, Artículo 180501. <https://doi.org/10.1007/s11432-020-2881-9>
- [2] Madanan, M., et al. (2023). Exploring Quantum Computing's Potential Breakthroughs and Challenges. *International Journal on Recent and Innovation Trends in Computing and Communication*, 11(11), 674–679. <https://doi.org/10.17762/ijritcc.v11i11.10070>
- [3] Nasser, A. (2025). Quantum Computing: Foundational and Theoretical Models. *Preprints*. <https://doi.org/10.20944/preprints202507.2057.v1>
- [4] Ozhigov, Y. I. (2022). Three principles of quantum computing. *Quantum Information and Computation*, 22(15-16), 1280–1288. <https://doi.org/10.26421/QIC22.15-16-2>
- [5] Singh Gill, S., Cetinkaya, O., Marrone, S., Caudino, D., Haunschild, D., Schlote, L., Wu, H., Ottaviani, C., & Liu, X. (2025). Quantum Computing: Vision and Challenges. En *Quantum Computing* (pp. 19–42). Morgan Kaufmann Publishers. <https://doi.org/10.1016/B978-0-443-29096-1.00008-8>
- [6] Farhi, E., Goldstone, J., & Gutmann, S. (2014). A Quantum Approximate Optimization Algorithm. *arXiv preprint arXiv:1411.4028*. <https://doi.org/10.48550/arXiv.1411.4028>
- [7] IBM Quantum. (2025). *Qiskit Runtime Primitives: Sampler V2*. IBM Corporation. Recuperado de <https://docs.quantum.ibm.com/api/qiskit-ibm-runtime>
- [8] Nielsen, M. A., & Chuang, I. L. (2010). *Quantum Computation and Quantum Information* (10th Anniversary ed.). Cambridge University Press. <https://doi.org/10.1017/CBO9780511>
- [9] Lov K. Grover.(1996). *A fast quantum mechanical algorithm for database search*. Cornell University. <https://doi.org/10.48550/arXiv.quant-ph/9605043>